

COSC 4P80 Neural Network

Nathan Le Dall
Brock University
Student Number: 7241250
Email: nl21mh@brocku.ca

Abstract—This report provides a detailed analysis of the performance of a back-propagation feedforward neural network. In their ability to correctly classify unseen data and their limitations. This report shows deferent improvements that can be made in order to improve accuracy.

I. INTRODUCTION

Feed forward neural networks (FNN) are a type of system that is made to imitate the human brain's ability to learn, find patterns, and make decisions. Feedforward neural networks can be given a set of inputs on which it has not been trained, and with a certain level of certainty it can predict the correct output for the given input. One use case of feedforward neural networks is that, as used in this report, to predict if a motor is good or defective based on data about the frequencies the motors output while being run. Their ability to do this is largely dependent on the size of the network, the amount of data given, and the viability of these data points.

II. BACKGROUND

Feedforward neural networks were initially proposed in 1943 by Warren McCulloch and Walter Pitts, these models are meant to imitate the structure of the brain, in having multiple layers of neurons each capable of giving slight input as to what the final output should be. over the years neural networks have evolved from single perceptrons to deep multilayer architectures capable of learning complex patterns.

Neural networks are usually represented as a set of matrices which allows for them to run in parallel allowing large models to be computed very quickly. Each matrix represents the weights between the layers of the network, which allows them to be computed using matrix operations. This allows for neural networks to be used and trained on moderns Graphics processing unit (GPUs), and Tensor processing unit (TPUs).

Training neural networks involves optimizing the value of weights and biases using algorithms such as back propagation which first computes how much error there is between the predicted output and the expected output using,

$$Err_p(W) = \frac{1}{2} \sum_{\Omega \in O} (o_{p,\Omega} - y_{p,\Omega})^2$$

The error is then corected and propagated back towards the rest of teh network using,

$$\Delta w_{i,\Omega} = -\eta \frac{dErr(W)}{dW_{i,\Omega}}$$

Depending on specific implementation, feedforward neural networks can complete a number of tasks, such as classification, image recognition, language translation. The design in this report attempts to classify motors as good or defective based on motor frequency data.

III. EXPERIMENTAL SETUP

Part A The network was implemented as a feed forward neural network with 3 layers, one input layer, one hidden layer, and one output layer. the input layer depends on which data file is used, there is one neuron per input so the 1000hz file has 10000 inputs. The hidden layer also uses a dynamic size, it is always half the size of the input layer. The output has 2 neurons 1 for good and one for defective. the main transfer functions used are ReLU, SIGMOID, and TanH. the learning rate was set to 0.000001 and is static as there are not enough epochs to warrant a dynamic rate. training was performed with 200 epochs, and using TanH. Convergence was encouraged by normalizing the input data, this helped the network have more statistically viable data.

Part B The input data was sectioned into 3 subsets which consisted of random data points within the data file, effectively implementing K-Fold-3. The proportion of good and bad motors was not forced to be consistent in each subset, however it was ensured that one data point was not used in multiple sets, for each fold two subsets were used for training and one was used for testing to ensure the network could not memorize the whole data file. the training was monitored by tracking the error at each epoch. The testing was monitored by tracking the accuracy of the network on the test data. experiments were conducted by changing the size of the network. Additionally the training and testing data were normalized in order to give more statistical validity to the data points.

Part C To improve training time, momentum was implemented into the back propagation algorithm. The same procedure from part B was run in an attempt to measure the difference in training speed. To prevent oscillation the momentum coefficient was moved to zero as the error got closer to 0.

Part D To determine which datasets or combination of datasets, yielded the best classification performance, each configuration cycled each dataset to make sure each possibility was tried.

Part E Building upon the previous parts, the network was expanded to 5 layers and the previous test were run again to compare performance. with a bigger architecture it was

expected for the network to be able to have better depth and therefore be able to recognize more patterns and have a better overall accuracy.

Part F Building on the extended network from the previous part, in Part F softmax was added to the last layer of neurons, the output, to see if having a different activation function for the last layer helped the accuracy of the network. the network was trained under identical situations to ensure the only changing factor was the activation function.

IV. RESULTS

Part B The results in Part B show that each data file performs similarly, while the best file is the 32 bin file that is most likely only due to the set up of the experiment. The 32 bin file most likely just barely has enough data to quickly train the small network to do better than the rest, while the the 1000 bin file likely over crowds it quickly. The 64 bin file trains the slowest likely due to the little amount of data there is, it is possible that this file specifically has data that is not favourable for the network to use.

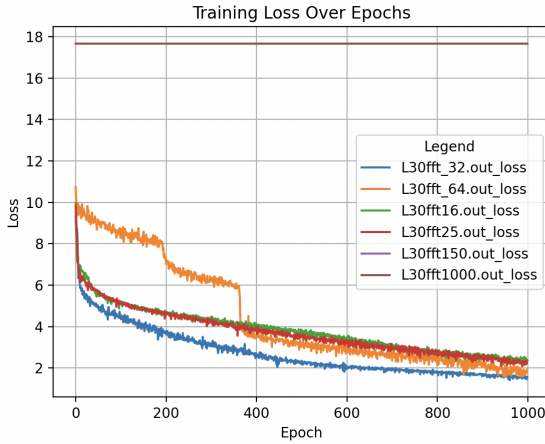


Fig. 1: Training loss curves for Part B using different FFT bin sizes.

Part C The graph for part C look very similar, and is likely identical to the graph from Part B. This is likely due to the fact that on such a small model the momentum does not have time to improve the training speed in any meaningful way, by the time the momentum starts to have an effect the training is over. On a larger model with hundreds of layers and more data, the momentum would have a great effect on training speed.

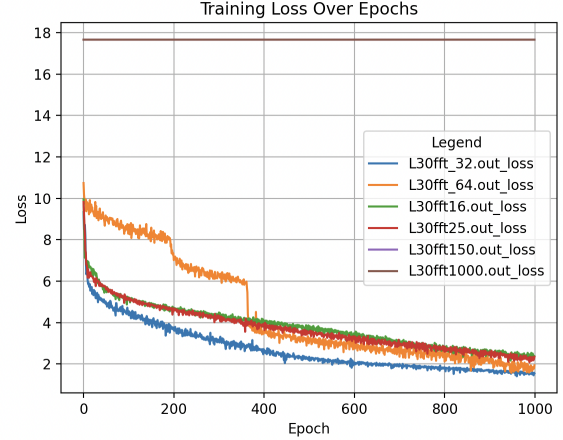


Fig. 2: Training loss curves for Part C under different FFT bin configurations.

Part D In Part D every file was broken into 3 subsets, each subset was excluded from the training set one after another to see which one was the best for each file. The table above

TABLE I: Best Loss Values for Each FFT Bin Configuration

FFT File	testing set 1	testing set 2	testing set 3
L30fft16.out	1.6677	3.3196	2.2010
L30fft_32.out	2.2261	0.3330	0.2146
L30fft_64.out	0.4905	1.0859	1.1238
L30fft25.out	2.2616	1.9651	0.2033
L30fft150.out	17.000	1.3681	0.0171
L30fft1000.out	0.0044	0.0016	0.0009

shows the best error for each dataset in each file. A lower score is better as it indicated the error is lower, while too low of an error is not good for neural networks, since training can be stopped early this table only displays how low the error can get not what error is best to use for a classification neural network.

Part E In part E the network was expanded to 5 layers, one input, one output, and 3 hidden layers. While this should improve the network's performance, in the scope of this report it did not, this could be for a number of reasons, the network does not have enough epochs to meaningfully improve, there is not enough data to fill a larger network, or the implementation does not reward a good error enough.

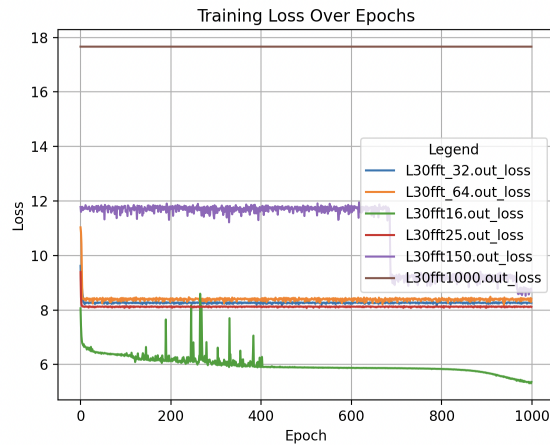


Fig. 3: Training loss curves for Part E

Part F In part F softmax was implemented, it is used to make sure the probabilities of each neuron add to one, this allows for the output to be more consistent and easier to classify. However, due to the last part not performing well, implementing softmax not only did not help it made everything worse. this is likely because since the model is not really training or improving softmax cannot get a consistent change on the output neurons to do anything, it does not matter how good softmax is if the network cannot produce anything close to the correct answer.

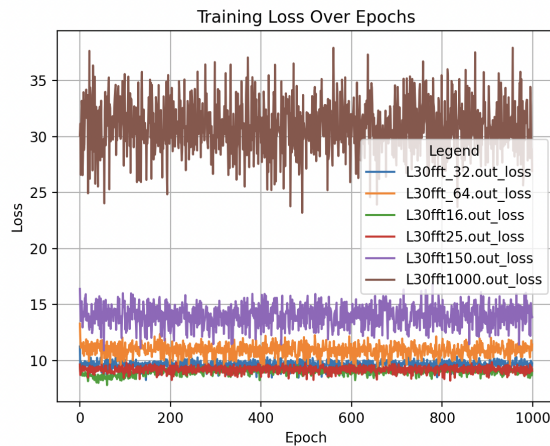


Fig. 4: Training loss curves for Part F.

V. CONCLUSION

In this study a feedforward neural network was implemented and tested to see how well it would perform. In Part A the architecture used was explained, in Part B K-Fold-3 was implemented to allow for better training and testing. In part C momentum was introduced to try to make the network learn faster. In Part D multiple data sets were combined in order to better represent the data. In Part E the layer was grown from 3 layers to 5. In Part F Softmax was implemented to allow

for the outputs to add to one making it easier to know which one the network picked. The results of the experiments may not all have been ones that were expected, however, they all showed the multitude of ways there are to improve a network's training speed and error

REFERENCES

- [1] P. D. Bockus, "Cosc 4p80: Artificial neural networks," Lecture slides, COSC 4P80, Brock University, 2025, accessed: 2025-11-03.
- [2] O. ChatGPT, "Chatgpt: Ai language model," <https://chat.openai.com/>, 2025, accessed: 2025-11-03.
- [3] N. LeDall, "Neural networks repository," <https://github.com/nathanLeDall/neural-networks>, 2025, accessed: 2025-11-03.